

COLLEGEMATE

Portfolio & Interview Case Study

Real-world Android MVP for academic coordination

Current status

Actively used MVP • 28/29 students • v1.1 (versionCode 2) • Firebase App Distribution

1. Executive Summary

CollegeMate is a real-world Android MVP built to solve academic-information fragmentation within a student batch. Important academic information was spread across WhatsApp conversations, PDF schedules, separate assignment/test messages, and other resources.

I designed and built CollegeMate using Kotlin, Jetpack Compose, reactive MVVM, Kotlin Flow/StateFlow, and Firebase. The project moved beyond local development into real deployment: the current release is v1.1 (versionCode 2), distributed through Firebase App Distribution, and 28 of 29 students are currently using it.

Rather than attempting to become another WhatsApp, CollegeMate focuses on academic organization: schedules, announcements, class cancellations, assignments, tests, academic resources, and notifications.

2. The Problem

The problem was not that academic information was unavailable. The problem was that students had to repeatedly search different places to find it.

Schedule: students often had to open PDF schedules just to answer “What class do I have today?”

Class changes: class cancellations and schedule changes could get buried among other WhatsApp messages.

Assignments and tests: information was distributed through conversations rather than maintained in a dedicated academic view.

Academic resources: syllabus, academic calendar, schedule, and related resources were scattered rather than organized around daily academic use.

3. Validation Before Building

I first conducted a survey of 30 students instead of assuming that another application was necessary.

These findings shaped the initial MVP around the highest-value academic workflows rather than attempting to build a broad college-management platform.

4. From Idea → Real MVP

The project followed a complete build-and-validate cycle:

Problem → Student survey → MVP definition → Android implementation → Firebase backend → Real-time synchronization → Push notifications → Release testing → Firebase App Distribution → Real students → Feedback

CollegeMate therefore represents a deployed product rather than a local-only prototype.

5. Product Today

Schedule: Today/Tomorrow/selected-date schedule, subject/faculty/time/classroom information, and personalized Open Elective resolution.

Class cancellation: Date-specific cancellations reflected directly on the affected schedule.

Announcements: Centralized academic announcement history.

Assignments & Tests: Deadlines/dates, details, and attachments in one organized view.

Notifications: FCM for remote academic notifications and WorkManager for scheduled local tasks such as birthday notifications.

6. Product Positioning

CollegeMate did not completely replace WhatsApp. Announcements currently run in parallel on WhatsApp and CollegeMate. Some students rely heavily on CollegeMate, some primarily use WhatsApp, and some use both.

This is a useful product distinction: CollegeMate focuses on structured academic information rather than general-purpose communication.

7. Why I Didn't Build Chat

Students suggested adding chat so CollegeMate could replace more of the WhatsApp experience. A general-purpose chat system would introduce real-time messaging infrastructure, message persistence, delivery/read states, notification handling, additional backend usage, and ongoing maintenance while largely reproducing functionality already provided by WhatsApp.

Decision: keep the MVP focused rather than expanding its feature surface simply because the feature was requested. Further development is currently paused because the expected return from additional CollegeMate work is lower than the return from investing that time in higher-priority technical growth.

Validation snapshot

Requirement	Survey result
Direct daily schedule view	93%
Assignments/tests organized in one place	93%
Dedicated app for class updates	63%

8. Actual Architecture

The final implementation is best described as a **reactive MVVM architecture with a Repository-based data layer**. The project does not use Clean Architecture, Hilt/Koin, or Room as part of the final implementation.

Firebase / Static Data → Repositories → Flow / StateFlow → ViewModels → Jetpack Compose UI

A reactive architecture means the UI observes changing application state rather than repeatedly requesting data manually. Repositories isolate Firebase data access, while ViewModels expose UI-ready state.

9. Application Data Flow

Authentication: Google Sign-In → AuthViewModel → AuthRepository → Firebase Authentication → institute-email validation → logged-in application.

Dynamic Firestore data: Firestore → addSnapshotListener → callbackFlow → Flow → ViewModel → StateFlow → Compose UI.

Schedule: selected date → RoutineResolver → static routine data + user-specific OE mapping → resolved schedule.

10. Engineering Story — Class Cancellation

The recurring timetable and a one-time cancellation represent two different kinds of data. Instead of modifying the recurring routine, CollegeMate stores cancellation information separately and overlays it at runtime.

Weekly Routine + Dynamic Firestore Cancellation → Runtime comparison → Normal ScheduleCard or CancelledScheduleCard

The student-side flow is: selected date → cancellation list filtered by date → each resolved class compared using identifiers such as subject code and start time → affected class rendered as cancelled.

This prevents a temporary cancellation from permanently modifying the recurring weekly routine.

11. Engineering Story — Assignment/Test & File Pipeline

Assignment/Test creation uses a four-step state-driven workflow: TYPE → SUBJECT → DETAILS → REVIEW → CONFIRM.

Attachment handling is separated from the main form state. Images are processed before upload; non-image documents such as PDFs bypass the image-optimization pipeline.

Image pipeline: original image → dimension inspection → downsampling → maximum 2048px dimension → EXIF orientation correction → JPEG compression at 80% → temporary optimized file → Firebase Storage → download URL → Firestore metadata.

The Firestore document is created only after the upload succeeds and the user confirms the final review. This reduces the chance of creating metadata that points to a failed upload.

12. Notification Architecture

Remote notifications: Firebase Cloud Messaging (FCM) receives remote payloads. Cancellation payloads are parsed and formatted into user-facing notifications.

Scheduled local notifications: WorkManager schedules background birthday checks, which can trigger local notifications.

The distinction is deliberate: FCM is used for remote academic events, while WorkManager handles scheduled local work.

13. Navigation & Network Awareness

Jetpack Compose Navigation provides route-based screen transitions. The navigation back stack is used as the source of truth for UI elements such as the TopBar and BottomBar.

ConnectivityManager → NetworkObserver → Flow → NetworkViewModel → MainView → Connectivity Banner

The network observer monitors internet capability and validated connectivity, allowing the application to communicate connection-state changes to the user.

14. Debugging Case Study — Samsung ModalBottomSheet Back Bug

Symptom: on a Samsung device, pressing Back while a text field and keyboard were active hid the keyboard and also dismissed the ModalBottomSheet. Other tested devices behaved normally.

Investigation: I first tested whether Samsung Keyboard was responsible, then changed navigation configurations, added logging, and experimented with conditional BackHandler/state-based solutions. These approaches did not explain the behavior.

Breakthrough: diagnostic logging showed that the first Back event was not reaching the normal Compose BackHandler path. Further tracing indicated that Material 3's ModalBottomSheet/window-level handling was consuming the event and triggering onDismissRequest.

Fix: disable automatic sheet dismissal with `shouldDismissOnBackPressed = false` and explicitly control dismissal with BackHandler.

Lesson: when an event behaves differently across devices, trace where the event is actually handled before changing application state.

15. Debugging Case Study — Release-only Google Sign-In Failure

Symptom: Google Sign-In worked in the debug build but failed in the release build.

The investigation compared package identity, release signing certificate SHA-1, Firebase/Google configuration, and API-key restrictions. Debug and release builds can represent different Android identities because their signing certificates differ.

The generic “Login Failed” message initially hid the underlying configuration error. Once the failure boundary was exposed, the investigation moved away from the Kotlin authentication flow and toward Google Cloud/Firebase configuration.

Reusable lesson: when debug works but release fails, compare the release identity and external configuration before rewriting application code.

16. Performance & Reliability Decisions

Client-side image processing: large images are resized, orientation-corrected, and compressed before upload. This reduces unnecessary data transfer and memory pressure.

Real-time synchronization: Firestore snapshot listeners were wrapped with `callbackFlow` so dynamic UI state updates without manual refresh.

Network awareness: connectivity state is surfaced to the UI through a dedicated observer/view-model path.

TTL cleanup: Firebase Console shows active Serving TTL policies for relevant `expiryDate` fields. The application-level expiry-date calculation still has a known limitation for some records.

17. Testing & Validation

The current project contains the standard Android JUnit and instrumentation test templates but does not contain a substantial project-specific automated test suite.

Validation instead relied heavily on manual testing, real-device testing, release testing, and actual MVP usage. The development process included online/offline checks, login/logout testing, assignment/test workflows, notification testing, class-cancellation testing, and device-specific debugging.

18. Deployment & Real-User Results

Initial validation: 30 students.

Real usage began: 16 August 2026.

Current active users: 28/29 students.

Distribution: Firebase App Distribution.

Current release: v1.1 / versionCode 2.

Status: Actively used MVP.

Firebase App Distribution independently shows the 1.1 (2) release on 16 August 2026, with 29 testers invited. The captured distribution state showed 24 accepted/downloaded at that point; this should not be interpreted as the current active-user count.

19. Real-User Feedback

Students particularly liked the Assignment/Test organization and the centralization of daily academic resources such as the syllabus and academic calendar.

They also requested chat, extra-class support, greater flexibility, and broader college-level deployment. These suggestions have been recorded, but major implementation of them is currently paused.

20. Limitations

Backend authorization: current Firebase rules distinguish authenticated from unauthenticated users but do not enforce separate student/CR roles.

Static routine data: the recurring timetable is largely represented in application-side seed data, which limits flexibility for college-wide deployment.

Automated testing: no meaningful project-specific automated test suite is currently present.

Expiry-date semantics: Firestore TTL is active, but the application-level expiry calculation has had a known limitation.

Orphaned uploads: a file can potentially remain in Firebase Storage if uploaded and then abandoned before the corresponding Firestore record is created.

Scalability: the current MVP is optimized for a controlled batch environment rather than a generalized college-wide deployment.

21. What I Would Change for College-Wide Scale

I would not simply add features to the current MVP. I would first redesign the data and authorization model around academic contexts such as college, department, batch, semester, students, CRs, and faculty.

The next-generation architecture would require stronger role-based authorization, dynamic timetable management, more comprehensive automated testing, controlled file ownership/access, lifecycle cleanup, and stronger operational observability.

These are future design directions, not claims about the current implementation.

22. What I Learned

Validate before building. The initial survey helped narrow the MVP to problems students actually cared about.

Separate stable data from temporary state. The cancellation overlay taught me to model recurring schedules and date-specific exceptions separately.

Debug the system, not just the code. The Samsung issue required tracing event dispatch; the Google Sign-In issue required tracing the release/configuration boundary.

Real devices reveal different problems. The Samsung issue was not obvious from normal development behavior.

Performance starts before the network. Image processing before upload reduces unnecessary transfer and memory usage.

Product development includes saying no. After deployment, I evaluated feature requests against their user value, engineering cost, infrastructure cost, and opportunity cost.

23. Current Status

CollegeMate is a completed, actively used MVP.

Problem → Validation → Design → Implementation → Debugging → Release → Real users → Feedback → Scope decision

The application remains operational and is currently used by 28/29 students. Major feature expansion is paused. The project therefore represents a complete build → deploy → validate → evaluate cycle rather than an unfinished prototype.

24. Technical Summary

Category	Implementation
Language	Kotlin
UI	Jetpack Compose + Material 3
Architecture	Reactive MVVM + Repository
State	Kotlin Flow / StateFlow
Database	Cloud Firestore
Authentication	Google Sign-In + Firebase Authentication
Storage	Firebase Storage
Notifications	Firebase Cloud Messaging + WorkManager
Navigation	Jetpack Compose Navigation
Distribution	Firebase App Distribution
Validation	30-student initial survey
Current usage	28/29 students
Current release	v1.1 / versionCode 2

25. One-Line Portfolio Description

Built and deployed CollegeMate, a Kotlin/Jetpack Compose academic coordination MVP validated with 30 students and currently used by 28/29 students, implementing reactive Firestore synchronization, date-specific class cancellation, Assignment/Test management with optimized file uploads, FCM notifications, and real-world Android debugging across release and device-specific issues.